

Практична робота №5

Виконали студенти групи ІП-44 Кулагін Дмитро та Комін Іван

Application: CreateSpace (Professional Space/Studio Booking for Creators)

Platform: Web

Тема: Масштабування системи

Мета: масштабувати кожен елемент системи

Завдання:

1. Зробити високорівневий capacity planning для кожного елементу системи (мікросервісу, бази даних, системи обміну повідомленнями тощо):
 - a. вказати навантаження, наприклад: скільки запитів може обслуговувати один інстанс мікросервісу, яке навантаження потягне база даних тощо
 - b. вказати відповідну конфігурацію сервера під навантаження
 - c. вказати вартість
2. Вкажіть для кожного елементу системи, який тип масштабування буде використовуватись в кожному випадку:
 - a. мікросервіси
 - b. система обміну повідомленнями
 - c. кеш (якщо є)
 - d. база даних
 - e. система потокової обробки даних (якщо є).
3. Вкажіть причини, з яких ви застосовуєте масштабування взагалі і кожний тип масштабування зокрема. Будьте дуже точними та базуйтеся на прикладі своєї системи та ваших кейсів.
4. Вказати для кожного елементу системи, як відбувається балансування навантаження у випадку горизонтального масштабування (як відбувається диспетчеризація)

Capacity planning

Для розрахунку **Capacity Planning** системи візьмемо за основу цільовий показник у **100,000 зареєстрованих користувачів** та пікове навантаження до **15,000 активних сесій одночасно**.

1. API Gateway (YARP)

Виступає єдиною точкою входу, відповідає за роутинг запитів та первинну перевірку JWT токенів.

- **Навантаження:** Оскільки кожен клієнтський запит проходить через шлюз, при піку в 15k користувачів очікується потік у **2,000 – 3,000 запитів за секунду (RPS)**. Один інстанс YARP на .NET 10 здатен обробляти такий об'єм при низькій затримці.
- **Конфігурація:** 2 vCPU, 4GB RAM (наприклад, AWS t3.medium або Azure B2s). Основне навантаження припадає на CPU через парсинг заголовків та валідацію підписів JWT.
- **Вартість:** ~\$25–30/міс за один інстанс.

2. Бізнес-мікросервіси (.NET Core)

Сюди входять Inventory, Scheduling, Billing та Identity. Кожен сервіс ізольований і виконує свою логіку.

- **Навантаження:** Inventory Service (Пошук): Найвище навантаження, до **1,000 RPS**, оскільки користувачі постійно застосовують фільтри.
 - Scheduling / Billing: Нижче навантаження (процес транзакцій), близько **200–400 RPS**.
- **Конфігурація:** 2 vCPU, 4GB RAM для кожного інстансу. ASP.NET Core має високу продуктивність, що дозволяє одному інстансу витримувати вказане навантаження.
- **Вартість:** ~\$30/міс за один інстанс кожного сервісу. Загалом для 5 доменів — ~\$150/міс.

3. Реляційні бази даних (PostgreSQL)

Використовується патерн **Database per Service**. Кожна БД логічно ізольована.

- **Навантаження:** Сумарно система має витримувати до **1,500 транзакцій за секунду (TPS)** на запис та значно більше на читання. Найбільш навантажена — БД домену Inventory (читання) та Billing (запис транзакцій).
- **Конфігурація:** 4 vCPU, 16GB RAM, SSD (наприклад, AWS RDS m5.xlarge). Велика кількість RAM критична для кешування індексів, щоб забезпечити швидкість пошуку < 500ms.
- **Вартість:** ~\$150–180/міс за один Managed інстанс (з урахуванням бекапів).

4. Message Broker (RabbitMQ)

Використовується для асинхронної обробки подій, таких як SMS-сповіщення та синхронізація календарів.

- **Навантаження:** Брокер повинен обробляти потік івентів (BookingConfirmedEvent, InventoryItemUpdated тощо) об'ємом до **5,000 – 10,000 повідомлень за секунду**.
- **Конфігурація:** 2 vCPU, 8GB RAM. Пам'ять важлива для утримання черг у моменти пікового навантаження або коли споживачі (Consumers) тимчасово недоступні.
- **Вартість:** ~\$70/міс.

5. Background Workers / Notification Service

Легковагові сервіси, що споживають івенти та взаємодіють із зовнішніми API.

- **Навантаження:** Залежить від швидкості зовнішніх сервісів (Twilio, Google Calendar). Очікувана швидкість обробки — **100–300 івентів/сек**.
- **Конфігурація:** 1 vCPU, 2GB RAM (Azure B1s).
- **Вартість:** ~\$15/міс за інстанс.

Типи масштабування

1. Мікросервіси (Inventory, Scheduling, Billing, Identity, Notifications)

- **Тип масштабування:** **Горизонтальне** (Horizontal Scaling / Scaling Out).
- **Як реалізується:** Додавання нових екземплярів кожного конкретного мікросервісу за допомогою Docker-контейнерів. Наприклад, при великій кількості запитів на пошук студій, система автоматично піднімає додаткові копії **Inventory Service**.
- **Обґрунтування:** Це критично для виконання нефункціональної вимоги щодо продуктивності — рендер пошукової видачі за < 500ms.

2. Система обміну повідомленнями (RabbitMQ)

- **Тип масштабування:** **Горизонтальне** (Clustering).
- **Як реалізується:** Створення кластера з декількох вузлів RabbitMQ. Використання **Quorum Queues** дозволяє реплікувати черги між вузлами.
- **Обґрунтування:** Забезпечує надійність — навіть якщо один вузол брокера вийде з ладу, івенти про бронювання не зникнуть. Це підтримує патерн **Dead Letter Queue** та **Retry Pattern**.

3. Кеш (Redis / Локальний кеш)

- **Тип масштабування:** Горизонтальне (Sharding / Clustering).
- **Як реалізується:** Розподіл даних між декількома вузлами кешу. У системі сервіси **Billing** та **Scheduling** використовують кеш для швидкого доступу до цін та статусів інвентарю.
- **Обґрунтування:** Дозволяє уникнути "пляшкового горла" при масових запитах на перевірку доступності обладнання під час пікових годин.

4. База даних (PostgreSQL)

- **Тип масштабування:** Гібридне (Вертикальне для запису + Горизонтальне для читання).
- **Як реалізується:**
 - **Вертикальне (Scaling Up):** Збільшення потужності (CPU, RAM) основного вузла (Primary) для обробки транзакцій у **Billing Service**.
 - **Горизонтальне (Read Replicas):** Створення реплік для читання, на які перенаправляються важкі запити пошуку та фільтрації з **Inventory Service**.
- **Обґрунтування:** Забезпечує цілісність даних (Data Integrity) для фінансових транзакцій при високому навантаженні на пошук. Оскільки використовується патерн **Database per Service**, кожна база масштабується незалежно.

5. Система потокової обробки даних (Stream Processing)

У межах масштабування системи **CreateSpace** розглядається можливість впровадження системи потокової обробки даних (наприклад, **Apache Flink** або **ksqlDB**) для розширення аналітичних можливостей платформи. Цей компонент є опціональним і призначений для сценаріїв, де звичайної асинхронної обробки через брокер повідомлень недостатньо, таких як

- **Аналітика в реальному часі:** Надання власникам студій миттєвої статистики щодо популярності їхніх локацій.
 - **Динамічне ціноутворення:** Автоматична корекція `pricePerHour` залежно від поточної інтенсивності пошукових запитів у певному регіоні.
 - **Виявлення аномалій:** Швидке реагування на підозрілі серії бронювань, що може свідчити про спроби зловживання платформою.
- **Тип масштабування:** Горизонтальне (Partitioning / Sharding).

- **Як реалізується:** **Apache Flink**, інтегрований із потоками даних від **RabbitMQ**. Система зчитує потік подій (наприклад, `InventoryItemUpdated` або `BookingCreated`), виконує віконні агрегації та зберігає результати в аналітичну БД або передає їх на UI-дашборди через **WebSockets**.
- **Обґрунтування:** Необхідно для обробки великих потоків подій у реальному часі, таких як масові оновлення доступності слотів через **Google Calendar API**.

Причини масштабування

1. Загальні причини застосування масштабування

Ми впроваджуємо масштабування для досягнення трьох ключових цілей, визначених у сценаріях атрибутів якості:

- **Продуктивність:** Гарантія того, що пошукова видача студій рендериться швидше, ніж за **500ms**, навіть при нормальному навантаженні.
- **Надійність:** Забезпечення роботи системи під час пікових годин бронювання, коли кількість одночасних запитів різко зростає.
- **Стійкість до збоїв:** Можливість обробляти івенти (наприклад, через **Retry Queue**), якщо зовнішні сервіси, як-от **Twilio**, тимчасово недоступні.

2. Причини застосування конкретних типів масштабування

Горизонтальне масштабування (Microservices & Message Broker)

- **Для мікросервісів:** Оскільки систему розділено на 5 незалежних доменів за принципом **Decomposition by Subdomain**, кожен сервіс має свій профіль навантаження.
 - **Кейс: Inventory Service** отримує набагато більше запитів на читання (пошук студій), ніж **Billing Service** на запис. Горизонтальне масштабування дозволяє нам запустити 10 інстансів пошуку і лише 2 інстанси білінгу, оптимізуючи витрати та ресурси.
- **Для RabbitMQ:** Ми використовуємо кластеризацію брокера для підтримки асинхронної комунікації.
 - **Кейс:** Якщо один вузол брокера вийде з ладу, івенти `BookingConfirmed` мають зберегтися в інших вузлах кластера, щоб **Notification Service** міг згодом надіслати SMS.

Вертикальне масштабування (Primary Database)

- **Для бази даних (Write Path):** Ми застосовуємо **Scale Up** (збільшення CPU/RAM) для основних інстансів **PostgreSQL**.
 - **Кейс:** Вимога **Data Integrity** вимагає рівно **0 double-bookings**. Найпростіший і найнадійніший спосіб забезпечити сувору консистентність при **Cross-Domain Checkout Calculation** — це потужний одиночний вузол бази даних, який швидко обробляє ACID-транзакції без затримок на мережеву синхронізацію між шардами.

Гібридне масштабування / Реплікація (Read Path)

- **Для бази даних (Inventory/Discovery):** Використання **Read Replicas**.
 - **Кейс:** Коли тисячі "Кріейторів" одночасно шукають студії, застосовуючи фільтри, ми переносимо це навантаження на репліки для читання. Це звільняє основну базу для критичних записів бронювань, забезпечуючи виконання вимоги щодо швидкості відгуку у **500ms**.

Балансування навантаження

Для системи **CreateSpace** балансування навантаження та диспетчеризація при горизонтальному масштабуванні відбуваються на декількох рівнях інфраструктури, забезпечуючи відмовостійкість та швидкість роботи.

1. Зовнішнє балансування (Клієнт → API Gateway)

Коли ми масштабуємо **API Gateway (YARP)** горизонтально, нам потрібна точка входу для розподілу зовнішнього трафіку від Web Application.

- **Механізм диспетчеризації:** Використовується хмарний балансувальник навантаження (L7 Load Balancer, наприклад, AWS ALB або Nginx).
- **Алгоритм:** Зазвичай **Round Robin** (почергово на кожен інстанс) або **Least Connections** (на той інстанс, де найменше активних сесій).

2. Балансування всередині системи (API Gateway → Мікросервіси)

Коли запит потрапляє в **YARP**, він має вирішити, на який саме екземпляр мікросервісу (наприклад, один із трьох інстансів Inventory Service) відправити запит.

- **Механізм диспетчеризації:** Вбудовані політики балансування в **YARP**.
- **Алгоритми: RoundRobin:** Запити рівномірно розподіляються між здоровими інстансами.
 - **PowerOfTwoChoices:** Вибираються два випадкові інстанси, і запит іде до того, який найменш завантажений.

- **Health Checks:** Gateway постійно опитує сервіси; якщо інстанс Scheduling Service не відповідає, він виключається зі списку диспетчеризації.

3. Асинхронна диспетчеризація (Message Broker → Споживачі)

У випадку масштабування **Notification Service** або **Billing Service**, брокер повідомлень сам керує розподілом роботи.

- **Механізм диспетчеризації:** Патерн **Competing Consumers** (Конкуруючі споживачі) у **RabbitMQ**.
- **Алгоритм:** RabbitMQ використовує **Round-robin distribution** для видачі повідомлень споживачам, що підключені до однієї черги.
- **Кейс:** Якщо в черзі BookingConfirmedEvent є 100 повідомлень і 2 інстанси сервісу сповіщень, кожен отримає по 50 задач паралельно.

4. Балансування бази даних (Read Replicas)

Оскільки ми використовуємо **Database per Service** та масштабуємо читання через репліки, нам потрібно розділяти потоки запитів.

- **Механізм диспетчеризації: Application-level:** У коді мікросервісу (наприклад, через EF Core) визначається, що запити GET (пошук студій) ідуть на Read-ReadOnly Connection String, а POST/PUT — на Primary.
 - **Proxy-level:** Використання **HAProxy** або **pgPool-II**, які аналізують SQL-запит і автоматично перенаправляють його на вільну репліку.

Висновок

У ході виконання роботи проведено планування потужностей (Capacity Planning) для системи **CreateSpace**, визначивши необхідні ресурси (vCPU, RAM) для функціонування **YARP API Gateway**, бізнес-мікросервісів та **PostgreSQL**. Обрано стратегію горизонтального масштабування для доменних сервісів та кластеризацію **RabbitMQ** для забезпечення високої доступності, а також вертикальне масштабування з використанням реплік для баз даних, що гарантує цілісність транзакцій та швидкість пошуку менше ніж **500ms**.

Реалізована багаторівнева диспетчеризація через **YARP** та патерн **Competing Consumers** у брокері повідомлень дозволяє ефективно розподіляти навантаження та забезпечувати стабільність системи при досягненні цільових показників у 100 000 користувачів.